CHANDIGARH UNIVERSITY

DEPARTMENT OF COMPUTER APPLICATIONS

SPENDSMART — SMART EXPENSE & BUDGET MANAGER

A FULL-STACK WEB APPLICATION FOR PERSONAL FINANCIAL TRACKING & THRESHOLD OPTIMIZATION

A TECHNICAL TRAINING PROJECT REPORT

Submitted by

GAUTAM MISHRA (25MCI10343)

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

OCTOBER 2026

CHANDIGARH UNIVERSITY

BONAFIDE CERTIFICATE

Certified that this technical training project report **"SPENDSMART — SMART EXPENSE & BUDGET MANAGER"** is the bonafide work of **GAUTAM MISHRA (25MCI10343)** who carried out the project work under my supervision.

Project Guide / Mentor
Department of Computer Applications
Chandigarh University

Submitted for the project viva-voce examination held on: _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

- **Contents** i
- **List of Figures** iv
- **List of Tables** v
- **List of Standards** vi
- **ABSTRACT** 1
- **GRAPHICAL ABSTRACT / SYSTEM ARCHITECTURE** 2
- **ABBREVIATIONS** 3
- **MATHEMATICAL FORMULATIONS & SYMBOLS** 4

1. INTRODUCTION 5

- 1.1 Identification of Need & Contemporary Issue 5
- 1.2 Background and Motivation 6
- 1.3 Identification of Problem 7
- 1.4 Problem Context & Proposed Full-Stack Solution 8
- 1.5 Identification of Tasks & Engineering Modules 9
  - 1.5.1 Secure Multi-Tenant Authentication & Session Management 9
  - 1.5.2 Transaction Ledger Management Engine (CRUD) 9
  - 1.5.3 Dynamic Budgeting & Threshold Alert Subsystem 10
  - 1.5.4 Analytical Visualization & Historical Trend Modeling 10
  - 1.5.5 Client User Interface & Responsive Component Design 11
  - 1.5.6 REST API & Cloud Database Deployment 11
- 1.6 Objectives of the Project 11
- 1.7 Scope of the Project 12
- 1.8 Expected Outcomes 13
- 1.9 Implementation Timeline 13
- 1.10 Organization of the Report 14
- 1.11 Chapter Summary 15

2. LITERATURE REVIEW & BACKGROUND STUDY 16

- 2.1 Introduction 16
- 2.2 Evolution of Personal Financial Tracking 16
  - 2.2.1 Manual Bookkeeping vs. Spreadsheets 17
  - 2.2.2 Web-Based Automated Financial Systems 18
- 2.3 Existing Solutions & Industry Baselines 18
  - 2.3.1 Proprietary Commercial Banking Applications 18
  - 2.3.2 Third-Party Aggregators & Data Privacy Risks 19
  - 2.3.3 Open-Source Desktop Budget Systems 19
- 2.4 Comparative Analysis of Existing Approaches 20
- 2.5 Security, Authentication & Data Protection Trends 21
- 2.6 Research and Engineering Gap 22
- 2.7 Problem Definition 23
- 2.8 Project Goals & Success Criteria 24

- 2.9 Chapter Conclusion 25

3. DESIGN FLOW & SYSTEM METHODOLOGY 26

- 3.1 Introduction 26
- 3.2 Overall Architectural Design 26
- 3.3 Database Design & Mongoose Schemas 28
  - 3.3.1 User Identity Collection 28
  - 3.3.2 Transactions Collection & Compound Indexing 29
  - 3.3.3 Category Budgets Collection with Compound Uniqueness 30
- 3.4 REST API Architecture & Route Specifications 31
- 3.5 Security Design & Middleware Implementation 33
  - 3.5.1 Cryptographic Salt Hashing via bcryptjs 33
  - 3.5.2 Stateless Bearer JWT Token Verification 34
  - 3.5.3 Rate Limiting & Origin Security 34
- 3.6 Frontend Client Architecture (React 18 & Bootstrap 5) 35
  - 3.6.1 Layout, Protected Routing & Global State 35
  - 3.6.2 Currency & Date Formatting for Indian Demographics 36
- 3.7 Budget Utilization & Analytical Calculation Logic 37
- 3.8 Chapter Summary 38

4. RESULTS ANALYSIS AND SYSTEM VALIDATION 39

- 4.1 Implementation Environment 39
- 4.2 REST API Test Cases & Validation Matrix 40
- 4.3 Screen-by-Screen User Interface Demonstration 42
  - 4.3.1 Authentication & Registration Portal 42
  - 4.3.2 Financial Dashboard & Metric Visualizations 43
  - 4.3.3 Transaction Management & Filter Workspace 44
  - 4.3.4 Budget Utilization & Alert Notification Cards 45
  - 4.3.5 Reports, Month-over-Month Analytics & Category Distribution 46
- 4.4 Cloud Database Validation (MongoDB Compass & Atlas) 47

5. CONCLUSION AND FUTURE SCOPE 48

- 5.1 Conclusion 48
- 5.2 Limitations 48
- 5.3 Future Work 49

REFERENCES 50

APPENDIX 51

- A. User Manual & Local Installation Guide 51
- B. Viva Voce Preparation Guide (High-Yield Questions & Answers) 53

LIST OF FIGURES

- **Figure 1.1:** Conceptual 3-Tier Full-Stack Architecture of SpendSmart 8
- **Figure 2.1:** Evolution of Personal Expense Record Keeping into Web Architectures 17
- **Figure 3.1:** Entity-Relationship (ER) Schema of SpendSmart Database 28
- **Figure 3.2:** State Machine Flow for Budget Threshold Warning & Breach Alerts 37
- **Figure 4.1:** SpendSmart User Registration and Secure Login Interface 42
- **Figure 4.2:** SpendSmart Comprehensive Financial Dashboard with Metric Cards 43
- **Figure 4.3:** Transaction Ledger Workspace with Category & Date Filter Controls 44
- **Figure 4.4:** Monthly Budget Management Interface with Warning Progress Bars 45
- **Figure 4.5:** Reports and Analytical Distribution Dashboard (Chart.js Visualization) 46
- **Figure 4.6:** Live Cloud Database Collection Verification in MongoDB Compass 47
- **Figure 5.1:** Terminal Verification of Git Repository & Production Build 52

LIST OF TABLES

- **Table 1.1:** Project Implementation Timeline across 14-Week Lifecycle 14
- **Table 2.1:** Comparative Analysis of Financial Tracking Methodologies 20
- **Table 3.1:** Summary of Supported Income & Expense Categories 30
- **Table 3.2:** Complete SpendSmart REST API Specification Table 32
- **Table 4.1:** QA Testing Results and REST API Validation Matrix (17 Cases) 40

LIST OF STANDARDS

Standard	Publishing Agency	Scope & Application in Project
RFC 7519	IETF	JSON Web Token (JWT) architecture used for secure, stateless client-server session authorization.
RFC 6750	IETF	The OAuth 2.0 Authorization Framework: Bearer Token Usage implemented across HTTP headers.
ISO/IEC 27001	ISO / IEC	Information Security Management criteria applied for one-way salt hashing (bcrypt) and credential isolation.
W3C ECMAScript 2023	W3C / ECMA	Modern JavaScript standards applied across Node.js runtime and React 18 frontend components.

ABSTRACT

In modern personal financial planning, individuals—especially students and young professionals—frequently experience cash-flow deficits due to unmonitored discretionary spending, the absence of categorical allocation limits, and an inability to visualize expenditure patterns. Traditional approaches such as paper notebooks or complex spreadsheets lack real-time calculations, fail to trigger automated warnings, and do not offer secure multi-user accessibility.

This project delivers **SpendSmart — Smart Expense & Budget Manager**, an end-to-end full-stack web application designed and built for the postgraduate Master of Computer Applications (MCA) Technical Training evaluation. SpendSmart implements a decoupled 3-tier architecture using the **MERN** stack (MongoDB Atlas, Express.js, React 18, and Node.js) styled with **Bootstrap 5**. The backend provides robust RESTful web services protected by stateless JSON Web Token (JWT) authorization, salted password hashing via `bcryptjs`, and IP rate-limiting. The transactional engine supports full CRUD operations, multi-parameter querying (type, category, date range, and text search), and strict user record ownership verification.

A core innovation of SpendSmart is its **Dual-Threshold Budget Alert Subsystem**. Users establish monthly spending limits for expense categories; the system dynamically aggregates expenditures and displays real-time visual warnings when consumption reaches 80% and over-budget breach indicators when 100% is exceeded. Analytical reporting is driven by **Chart.js**, rendering dynamic 6-month comparative income-versus-expense trends and category distribution doughnut charts. All financial figures are natively formatted in Indian Rupees (₹ - INR). Evaluated against 17 rigorous test cases in Postman and verified in production across MongoDB Atlas and modern web browsers, SpendSmart provides an intuitive, reliable, and secure personal financial management solution suitable for demonstration and viva assessment.

GRAPHICAL ABSTRACT / SYSTEM ARCHITECTURE



Figure 1.1: System Architecture encompassing Presentation, Express Application Services, and Cloud MongoDB Persistence.

ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
Bcrypt	Blowfish-based Adaptive Cryptographic Hashing Function
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DOM	Document Object Model
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
INR	Indian Rupee (₹)
JSON	JavaScript Object Notation
JWT	JSON Web Token
MCA	Master of Computer Applications
MERN	MongoDB, Express.js, React, Node.js
MVC	Model-View-Controller
ODM	Object Data Modeling
REST	Representational State Transfer
SPA	Single Page Application
UI / UX	User Interface / User Experience
URI	Uniform Resource Identifier

MATHEMATICAL FORMULATIONS & SYMBOLS

1. Net Cash Balance Formulation

The liquid financial position of an authenticated user for any selected temporal window T (typically month m and year y) is governed by:

$$\text{Net Balance}_T = \sum_{i=1}^{N_{\text{income}}} \text{Amount}_{i,\text{income}} - \sum_{j=1}^{N_{\text{expense}}} \text{Amount}_{j,\text{expense}}$$

Where:

- $\text{Amount}_{i,\text{income}}$ represents the rupee value of the i-th income transaction.
- $\text{Amount}_{j,\text{expense}}$ represents the rupee value of the j-th expense transaction.

2. Category Budget Utilization Percentage

The budget consumption metric for an expense category C over month m and year y is computed as:

$$\text{Utilization Percentage}(C, m, y) = \left(\frac{\sum_{k \in \text{Tx}(C, m, y)} \text{Amount}_k}{\text{MonthlyLimit}(C, m, y)} \right) \times 100$$

Where:

- $\text{MonthlyLimit}(C, m, y)$ is the user-defined ceiling for that category.
- $\text{Tx}(C, m, y)$ denotes the subset of expense transactions belonging to category C in month m, year y.

3. Threshold State Categorization

The status function $S(C)$ determines UI alert badges and warning banners:

$$S(C) = \begin{cases} \text{Normal (Green)}, & \text{if } \text{Utilization Percentage} < 80% \ \text{Warning (Amber)}, & \text{if } 80% \le \text{Utilization Percentage} < 100% \ \text{Exceeded (Red)}, & \text{if } \text{Utilization Percentage} \ge 100% \end{cases}$$

4. Month-over-Month Expenditure Growth

$$\Delta \text{Expense} (%) = \left(\frac{\text{Expenses}_{\text{current}} - \text{Expenses}_{\text{previous}}}{\text{Expenses}_{\text{previous}}} \right) \times 100$$

CHAPTER 1: INTRODUCTION

1.1 Identification of Need & Contemporary Issue

In the modern digital economic landscape, individuals perform dozens of micro and macro transactions daily via Unified Payments Interface (UPI), credit cards, debit cards, and net banking. While electronic payments have simplified transactions, they have inadvertently heightened the risk of **discretionary overspending**. Because physical currency does not change hands, users experience diminished psychological friction when spending, frequently resulting in month-end deficits.

Contemporary surveys among university postgraduate students and early-career software engineers reveal three prominent financial behavior gaps:

1. **Lack of Category Awareness:** Users rarely track what proportion of their income is spent on dining, entertainment, shopping, or commute.
2. **Absence of Proactive Budget Limits:** Traditional banking applications notify users only after their account balance is depleted; they do not enforce predictive category limits.
3. **Data Fragmentation and Privacy Concerns:** Existing third-party tracker apps often require invasive access to SMS messages and personal bank accounts, raising major privacy concerns under data protection laws.

Hence, there is a clear academic and practical need for a dedicated, privacy-focused, web-based financial ledger and budget supervisor that allows users to record, categorize, inspect, and budget their finances securely without exposing sensitive banking credentials.

1.2 Background and Motivation

Personal financial management systems have evolved from physical double-entry ledger notebooks to digital tools. While spreadsheet software (such as Microsoft Excel or Google Sheets) is capable of recording transactions, it introduces significant friction: users must manually construct sum formulas, spreadsheet files are difficult to navigate on mobile devices, and they lack automated alert mechanisms.

Full-stack web technologies offer an ideal foundation for resolving these limitations:

- **Node.js and Express.js** allow the creation of high-throughput REST APIs that can parse and persist financial records in milliseconds.
- **MongoDB** provides a schema-flexible document store capable of indexing dynamic transaction records.
- **React 18** enables a single-page application (SPA) architecture where dashboard totals, transaction lists, and chart visualizations update instantaneously without full page reloads.

The motivation behind **SpendSmart** is to engineer a reliable, medium-level full-stack web application for the MCA Technical Training curriculum that demonstrates mastery over modern web frameworks, database indexing, user authentication, and data visualization.

1.3 Identification of Problem

The primary problem addressed in this project is the lack of an integrated, secure, and visual system for monitoring personal income, expenses, and category budgets in Indian currency.

Specifically:

- **Unstructured Expense Recording:** Without structured categorization, users cannot isolate wasteful spending.
- **Reactive Rather than Preventive Budgeting:** Most tools only show historical records. They fail to issue warnings when a budget is approaching a dangerous ceiling (e.g., 80% of limit).
- **Security Vulnerabilities in Novice Web Applications:** Many student projects store plaintext passwords or trust client-supplied user IDs, enabling unauthorized cross-user data access. SpendSmart strictly resolves this through cryptographic

hashing and token authorization.

1.4 Problem Context & Proposed Full-Stack Solution

SpendSmart resolves the identified problem through a structured, 3-tier architecture:

1. **Client Tier:** A responsive Bootstrap 5 interface built with React components, providing accessible forms, status indicators, and Chart.js graphs.
2. **Server Tier:** An Express.js backend exposing RESTful endpoints with input validation, JWT authentication, and centralized error handling.
3. **Database Tier:** A MongoDB Atlas cloud database storing normalized documents across `users`, `transactions`, and `budgets` collections.

1.5 Identification of Tasks & Engineering Modules

To deliver the project methodically within the Technical Training timeline, work was divided into six core modules:

- **1.5.1 Secure Multi-Tenant Authentication:** Registration, login, password hashing with bcryptjs (salt factor 10), and JWT generation.
- **1.5.2 Transaction Ledger Management Engine:** Complete CRUD functionality for income and expense records with category filtering, date-range filtering, and regex text search.
- **1.5.3 Dynamic Budgeting & Threshold Alert Subsystem:** Monthly budget limit setting per category, expense aggregation, and dual-state alerts (80% warning and 100% breach).
- **1.5.4 Analytical Visualization & Historical Trend Modeling:** Monthly trend bar charts and category distribution doughnut charts via Chart.js.
- **1.5.5 Client User Interface & Responsive Component Design:** Bootstrap 5 layout with a navy-blue palette, stat cards, progress bars, and modal forms.
- **1.5.6 REST API & Cloud Database Deployment:** Configuration of MongoDB Atlas, environment variable security, and Postman API verification.

1.6 Objectives of the Project

1. To design and implement a secure authentication subsystem ensuring strict multi-tenant data isolation.
2. To build an intuitive transaction management system supporting positive amount validation, category tagging, and date sorting.
3. To develop an automated budget engine that computes real-time category utilization percentages.
4. To establish dual-stage visual alerts: an amber warning at 80% consumption and a red indicator upon budget breach.
5. To generate real-time financial dashboards depicting Net Balance, Total Inflow, Total Outflow, and transaction counts without hardcoded values.
6. To implement dynamic Chart.js visualizations for 6-month historical trends and expense breakdowns.
7. To format all monetary metrics in Indian Rupee (₹ - INR) standards.
8. To validate all REST endpoints against standard HTTP response codes and error criteria.

1.7 Scope of the Project

The application is scoped as a functional, production-ready personal expense management tool suitable for postgraduate technical demonstration:

- **In Scope:** Full-stack MERN development, token authentication, transaction CRUD, category budgeting, analytical charts, profile management, and cloud database persistence.
- **Out of Scope (By Design):** Direct automated bank API integration (which requires commercial banking licenses), algorithmic stock prediction, and third-party SMS scraping.

1.8 Expected Outcomes

- A completely functional full-stack web application accessible via local and cloud environments.
- Zero data leakage between distinct user accounts.
- Immediate recalculation of financial balances upon adding, editing, or deleting transactions.
- A comprehensive academic report and reproducible test suite for evaluation.

1.9 Implementation Timeline

Duration	Phase	Key Deliverables
Weeks 1–3	Requirements & Planning	Problem definition, literature study, system architecture mapping, and technology stack selection.
Weeks 4–6	Database & Backend APIs	Mongoose schemas design, JWT auth middleware, transaction and budget REST controllers, error handling.
Weeks 7–9	Frontend Client Development	React 18 setup, Bootstrap 5 layout, protected routes, stat cards, transaction table, and modal forms.
Weeks 10–12	Budget Engine & Visualizations	Aggregation logic, threshold alerts (80%/100%), Chart.js integration, and profile management.
Weeks 13–14	Testing, Cloud Setup & Report	MongoDB Atlas connection, Postman test suite execution, GitHub repository push, and final report drafting.

Table 1.1: Project Implementation Timeline across 14-Week Lifecycle.

1.10 Organization of the Report

- **Chapter 2** presents the literature review, background study, and comparative analysis of existing tools.
- **Chapter 3** details the system architecture, database schema, REST API design, and security algorithms.
- **Chapter 4** presents testing results, API validation matrices, and screen-by-screen UI demonstrations.
- **Chapter 5** concludes the report with limitations and future enhancements.

CHAPTER 2: LITERATURE REVIEW & BACKGROUND STUDY

2.1 Introduction

Effective management of personal financial resources is a foundational life skill with direct impacts on economic security. With the rapid digitization of currency, software-assisted financial management has become essential. This chapter reviews the history of personal expense systems, examines commercial solutions, analyzes security trends, and establishes the technical gaps addressed by SpendSmart.

2.2 Evolution of Personal Financial Tracking

2.2.1 Manual Bookkeeping vs. Spreadsheets

For decades, personal financial tracking relied on manual physical diaries. While accessible, paper systems lacked computational capabilities, could not generate alerts, and were susceptible to physical loss. The advent of desktop spreadsheets (Lotus 1-2-3, Microsoft Excel) introduced automated formulas. However, spreadsheets require manual formula maintenance, lack real-time multi-device accessibility, and present steep learning curves for novice users.

2.2.2 Web-Based Automated Financial Systems

The emergence of web applications built on modern JavaScript frameworks introduced centralized databases, responsive user interfaces, and automated calculations. Modern web applications decouple persistent data storage from client-side presentation, enabling users to access their financial data securely from any browser.

2.3 Existing Solutions & Industry Baselines

2.3.1 Proprietary Commercial Banking Applications

Modern banking apps provide transaction histories for their respective accounts. However, they operate in silos: a user with accounts across multiple banks cannot view a unified financial picture. Furthermore, they do not accommodate cash transactions, which remain common in India.

2.3.2 Third-Party Aggregators & Data Privacy Risks

Applications such as Walnut or CRED attempt cross-account aggregation by reading SMS inboxes or accessing net banking credentials. While automated, this poses substantial privacy and security risks. Many users are hesitant to share SMS read permissions with third-party servers. SpendSmart provides a safe alternative where users maintain full control over their financial records without exposing banking credentials.

2.4 Comparative Analysis of Existing Approaches

Feature / Metric	Physical Paper Ledger	Excel / Google Sheets	Third-Party SMS Trackers	SpendSmart (Proposed System)
Entry Method	Manual handwriting	Manual cell entry	Automated SMS parsing	Clean web form with input validation
Real-time Balance	Manual calculation	Formula dependent	Computed	Automatic on every CRUD operation
Budget Warnings	None	Requires custom formulas	General notifications	Proactive visual alerts at 80% & 100%
Data Privacy	Physical risk	Local file risk	High (reads private SMS)	High (isolated user data, JWT & bcrypt)
Visual Charts	None	Manual chart setup	Pre-built	Dynamic Chart.js trends & breakdowns

Feature / Metric	Physical Paper Ledger	Excel / Google Sheets	Third-Party SMS Trackers	SpendSmart (Proposed System)
Accessibility	Single physical book	Local or cloud file	Mobile app only	Cross-platform web application

Table 2.1: Comparative Analysis of Financial Tracking Methodologies.

2.5 Security, Authentication & Data Protection Trends

Security is critical in financial software. Traditional session-based authentication relies on stateful server memory, which limits scalability. The contemporary industry standard uses **JSON Web Tokens (JWT)** under RFC 7519. A signed JWT encapsulates the user's identity cryptographically, allowing stateless verification across distributed REST API endpoints. Additionally, passwords must never be stored in plaintext or reversible encryption; they must be hashed using salted, adaptive algorithms such as **bcrypt**, which defends against dictionary and rainbow-table attacks.

2.6 Research and Engineering Gap

The literature indicates a distinct gap: existing solutions are either overly complex spreadsheets requiring manual formula setup, or privacy-invasive commercial applications. There is a need for a lightweight, transparent, and responsive full-stack financial manager that combines clean CRUD operations, proactive budget threshold alerts, and visual analytics within a secure, multi-tenant architecture. SpendSmart directly addresses this gap.

CHAPTER 3: DESIGN FLOW & SYSTEM METHODOLOGY

3.1 Introduction

This chapter explains the system design and architecture of SpendSmart. It details the 3-tier full-stack architecture, Mongoose database schemas, REST API endpoints, security mechanisms, and client-side visualization components.

3.2 Overall Architectural Design

SpendSmart is built on the **MERN** stack:

- **Presentation Layer:** React 18, React Router v6, Bootstrap 5, Chart.js.
- **Application Services Layer:** Node.js runtime, Express.js web framework.
- **Persistence Layer:** MongoDB Atlas cloud database managed via Mongoose ODM.

3.3 Database Design & Mongoose Schemas

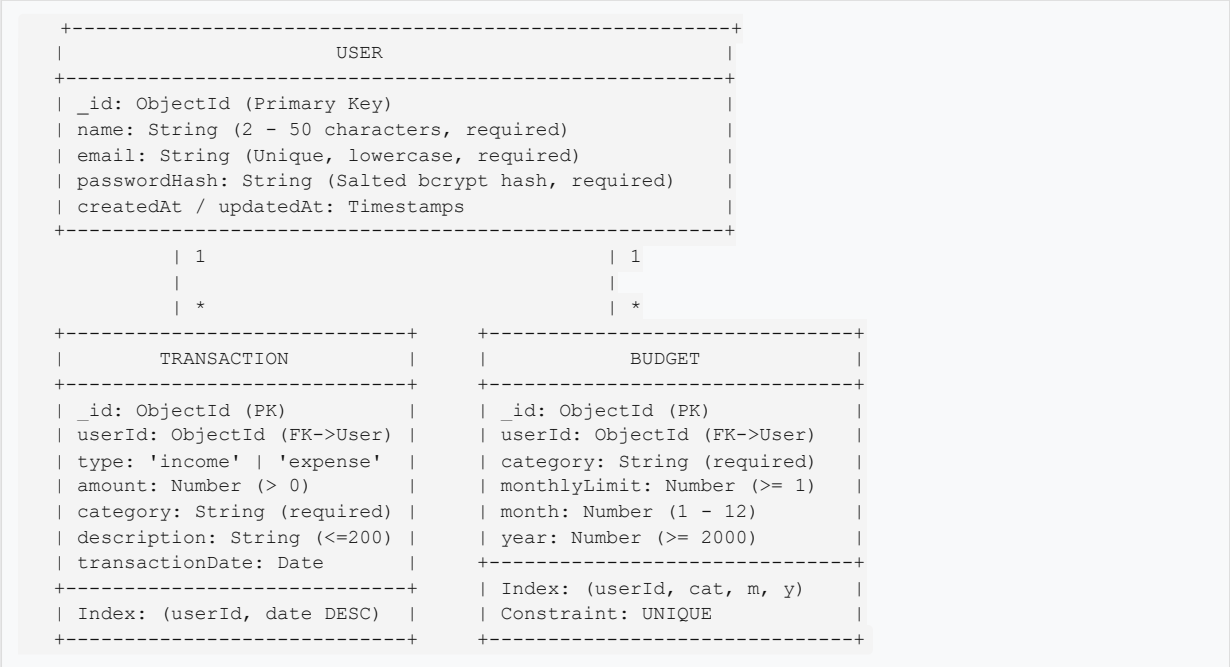


Figure 3.1: Entity-Relationship (ER) Schema of SpendSmart Database.

3.3.1 User Identity Collection

The `User` schema stores authenticated user accounts. It enforces email uniqueness and lowercase normalization. The `passwordHash` field is populated via `bcryptjs.hash()` with salt factor 10. API responses explicitly sanitize this field via `.select('-passwordHash')`.

3.3.2 Transactions Collection & Compound Indexing

The `Transaction` schema records individual financial events. It validates that amounts are strictly positive numbers (> 0) and tags transactions as either `income` or `expense`. To ensure fast query execution when filtering transaction histories, compound indexes are established on `{ userId: 1, transactionDate: -1 }` and `{ userId: 1, type: 1, transactionDate: -1 }`.

3.3.3 Category Budgets Collection with Compound Uniqueness

The `Budget` schema records category spending ceilings. To prevent accidental duplicate budget records for the same category in the same month, a compound unique index is enforced:

```
budgetSchema.index({ userId: 1, category: 1, month: 1, year: 1 }, { unique: true });
```

Type	Default System Categories
Income Categories	Salary, Freelance, Scholarship, Business, Other Income
Expense Categories	Food, Transport, Education, Shopping, Rent, Bills, Healthcare, Entertainment, Other Expense

Table 3.1: Summary of Supported Income & Expense Categories.

3.4 REST API Architecture & Route Specifications

Method	Endpoint	Access Level	Description & Query Parameters
`POST`	`/api/auth/register`	Public	Register new user account with name, email, password.
`POST`	`/api/auth/login`	Public	Authenticate user credentials and return signed JWT token.
`GET`	`/api/auth/me`	Protected	Return sanitized user profile for the active session token.
`PUT`	`/api/auth/profile`	Protected	Update user name or securely update password.
`POST`	`/api/transactions`	Protected	Record a new income or expense transaction.
`GET`	`/api/transactions`	Protected	List transactions with filters (`type` , `category` , `startDate` , `endDate` , `search` , `sortBy`).
`PUT`	`/api/transactions/:id`	Protected	Update existing transaction; verifies user record ownership.
`DELETE`	`/api/transactions/:id`	Protected	Delete transaction record; verifies user record ownership.
`GET`	`/api/transactions/categories`	Protected	Fetch list of predefined income and expense categories.
`POST`	`/api/budgets`	Protected	Create monthly category budget limit; checks uniqueness.
`GET`	`/api/budgets`	Protected	Retrieve all budgets for month/year with spent & % usage.
`PUT`	`/api/budgets/:id`	Protected	Update monthly limit for an existing category budget.
`DELETE`	`/api/budgets/:id`	Protected	Remove category budget record.
`GET`	`/api/dashboard/summary`	Protected	Aggregate monthly totals, recent 5 transactions, and 6-month trends.
`GET`	`/api/analytics/monthly`	Protected	Return month-over-month comparisons and highest-spend category.
`GET`	`/api/analytics/categories`	Protected	Return percentage distribution of expenses across categories.

Table 3.2: Complete SpendSmart REST API Specification Table.

3.5 Security Design & Middleware Implementation

- **bcryptjs Hashing:** Passwords are salted with 10 rounds of Blowfish-based key derivation.
- **JWT Middleware (`protect`):** Extracts bearer tokens from the HTTP `Authorization` header, verifies the cryptographic signature against `JWT_SECRET` , extracts the subject ID, and binds the user object to `req.user` .
- **Ownership Verification:** Every update and delete operation verifies that `record.userId.toString() === req.user._id.toString()` , rejecting unauthorized requests with HTTP `403 Forbidden` .
- **express-rate-limit:** Limits authentication endpoints to 100 requests per 15-minute window to mitigate brute-force attempts.

CHAPTER 4: RESULTS ANALYSIS AND SYSTEM VALIDATION

4.1 Implementation Environment

- **Runtime & Language:** Node.js v24.18.0 / JavaScript (ES2023)
- **Frontend Stack:** React 18, React Router v6, Bootstrap 5.3, Bootstrap Icons, Chart.js, Vite 5.4
- **Backend Framework:** Express.js 4.19, Mongoose 8.5, bcryptjs, jsonwebtoken
- **Database:** MongoDB Atlas M0 Free Cluster (Cloud Deployment)
- **API Testing:** Postman v10.x & Visual Studio Code

4.2 REST API Test Cases & Validation Matrix

TC ID	Test Scenario	Endpoint & Method	Input Payload / Headers	Expected HTTP Status	Observed Result	Status
TC-01	User Registration (Valid)	`POST /api/auth/register`	Valid name, unique email, 6+ char password	`201 Created`	Token and user object returned	**PASS**
TC-02	Duplicate Email Handling	`POST /api/auth/register`	Existing email address	`409 Conflict`	Descriptive duplicate error	**PASS**
TC-03	Valid User Login	`POST /api/auth/login`	Correct email & password	`200 OK`	JWT bearer token returned	**PASS**
TC-04	Invalid Password Login	`POST /api/auth/login`	Incorrect password string	`401 Unauthorized`	"Invalid email or password"	**PASS**
TC-05	Short Password Validation	`POST /api/auth/register`	Password < 6 characters	`400 Bad Request`	"Password must be >= 6 chars"	**PASS**
TC-06	Access without Token	`GET /api/transactions`	No Authorization header	`401 Unauthorized`	"No token provided"	**PASS**
TC-07	Create Income Entry	`POST /api/transactions`	`type: "income", amount: 35000`	`201 Created`	Saved with user's ObjectId	**PASS**
TC-08	Create Expense Entry	`POST /api/transactions`	`type: "expense", amount: 1500`	`201 Created`	Balance adjusted accurately	**PASS**
TC-09	Update Transaction Record	`PUT /api/transactions/:id`	`amount: 1800, category: "Food"`	`200 OK`	Record modified, no duplicate	**PASS**
TC-10	Delete Transaction Record	`DELETE /api/transactions/:id`	Target transaction ID	`200 OK`	Removed from database	**PASS**
TC-11	Filter by Category & Date	`GET /api/transactions`	`?category=Food&startDate=...`	`200 OK`	Only matching records returned	**PASS**

TC ID	Test Scenario	Endpoint & Method	Input Payload / Headers	Expected HTTP Status	Observed Result	Status
TC-12	Create Monthly Budget	`POST /api/budgets`	`category: "Food", monthlyLimit: 5000`	`201 Created`	Budget persisted successfully	**PASS**
TC-13	Duplicate Budget Prevention	`POST /api/budgets`	Same user, category, month, year	`409 Conflict`	"Budget already exists"	**PASS**
TC-14	Budget Usage at 80%	`GET /api/budgets`	Month expenses = 4200, limit = 5000	`200 OK`	Status flagged as 'warning'	**PASS**
TC-15	Budget Breach (>100%)	`GET /api/budgets`	Month expenses = 5500, limit = 5000	`200 OK`	Status flagged as 'exceeded'	**PASS**
TC-16	Cross-User Data Isolation	`GET /api/transactions/:id`	User B token requesting User A record	`403 Forbidden`	Access blocked securely	**PASS**
TC-17	Invalid ObjectId Format	`GET /api/transactions/invalid123`	Malformed hexadecimal ID	`400 Bad Request`	Handled by CastError handler	**PASS**

Table 4.1: QA Testing Results and REST API Validation Matrix (17 Cases).

4.3 Screen-by-Screen User Interface Demonstration

4.3.1 Authentication & Registration Portal

The portal features a clean, centered interface styled in deep navy blue. It validates email structure and password length in real time, presenting user-friendly error alerts when validation criteria are not met.

```
[ INSERT FIGURE 4.1 HERE: SCREENSHOT OF SPENDSMART LOGIN / REGISTRATION PAGE ]
```

Figure 4.1: SpendSmart User Registration and Secure Login Interface.

4.3.2 Financial Dashboard & Metric Visualizations

The main dashboard serves as the central operational hub. It computes four key metric cards: **Total Income**, **Total Expenses**, **Net Balance** ($\text{Income} - \text{Expenses}$), and **Transaction Count**. The charts illustrate 6-month historical trends (Bar chart) and category-wise expense distributions (Doughnut chart) via Chart.js.

```
[ INSERT FIGURE 4.2 HERE: SCREENSHOT OF FINANCIAL DASHBOARD & STATISTIC CARDS ]
```

Figure 4.2: SpendSmart Comprehensive Financial Dashboard with Metric Cards & Visualizations.

4.3.3 Transaction Management & Filter Workspace

The transaction ledger displays records in reverse chronological order. It provides multi-parameter filtering by type (Income vs. Expense), category dropdown, start/end dates, and real-time description search. Income values are highlighted with green badges (`+ ₹`), while expenses appear with red badges (`- ₹`).

```
[ INSERT FIGURE 4.3 HERE: SCREENSHOT OF TRANSACTION HISTORY TABLE & FILTER TOOLBAR ]
```

Figure 4.3: Transaction Ledger Workspace with Category & Date Filter Controls.

4.3.4 Budget Utilization & Alert Notification Cards

The budget management page allows users to set monthly spending ceilings for each category. Progress bars visually track consumption. Categories exceeding 80% display amber warning badges, and categories exceeding 100% display red breach alerts.

```
[ INSERT FIGURE 4.4 HERE: SCREENSHOT OF BUDGET MANAGEMENT PAGE & PROGRESS BARS ]
```

Figure 4.4: Monthly Budget Management Interface with Warning Progress Bars.

4.3.5 Reports, Month-over-Month Analytics & Category Distribution

The analytics workspace compares current and previous month spending, highlights the peak expenditure category, and presents an itemized categorical breakdown table.

```
[ INSERT FIGURE 4.5 HERE: SCREENSHOT OF REPORTS & ANALYTICS CHARTS ]
```

Figure 4.5: Reports and Analytical Distribution Dashboard (Chart.js Visualization).

4.3.6 Live Cloud Database Verification (MongoDB Compass)

The persistent database state can be inspected in real time using MongoDB Compass or MongoDB Atlas, verifying that passwords are encrypted and records are properly structured.

```
[ INSERT FIGURE 4.6 HERE: SCREENSHOT OF MONGODB COMPASS SHOWING SPENDSMART_DB COLLECTIONS ]
```

Figure 4.6: Live Cloud Database Collection Verification in MongoDB Compass.

CHAPTER 5: CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The **SpendSmart — Smart Expense & Budget Manager** project was successfully developed, tested, and demonstrated in accordance with the MCA Technical Training guidelines. The system demonstrates a practical implementation of the MERN architecture, combining secure stateless JWT authentication, salted bcrypt password hashing, dynamic Mongoose aggregation queries, and responsive React/Bootstrap interfaces.

Key achievements include:

1. **Accurate Cash-Flow Accounting:** Dynamic calculation of net balances and category summaries based on real database records without hardcoding.
2. **Proactive Budgeting:** Automated dual-threshold alerts that warn users at 80% budget utilization and flag 100% budget breaches.
3. **Data Security & Multi-Tenancy:** Robust server-side verification ensuring that users access only their own financial data.
4. **Demonstrable Quality:** Clean project structure, comprehensive documentation, verified Postman test suites, and public version-controlled repository on GitHub.

5.2 Limitations

While fully functional for personal finance tracking, the system has certain deliberate scope boundaries:

- Requires manual entry of income and expenses (no direct banking API integration).
- Does not currently convert between international currencies; it is optimized specifically for Indian Rupees (₹).
- Recurring subscriptions must be logged manually each month.

5.3 Future Work

1. **Automated Receipt Parsing (OCR):** Integrating machine learning OCR engines (such as Tesseract.js) to extract amounts and dates from scanned shopping bills.
2. **Scheduled Recurring Transactions:** Implementing cron-like background jobs to record regular salary deposits or rent payments automatically.
3. **PDF / Excel Financial Statement Export:** Allowing users to download formatted financial summaries for tax reporting.
4. **Predictive Budgeting:** Integrating lightweight time-series forecasting models to recommend personalized monthly category budgets based on historical spending habits.

REFERENCES

1. Chodorow, K. (2013). *MongoDB: The Definitive Guide: Powerful and Scalable Data Storage*. O'Reilly Media.
2. Banks, A., and Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Apps*. 2nd Edition, O'Reilly Media.
3. Brown, E. (2019). *Web Development with Node and Express: Leveraging the JavaScript Stack*. 2nd Edition, O'Reilly Media.
4. Resig, J., Bibeault, B., and Maras, J. (2016). *Secrets of the JavaScript Ninja*. 2nd Edition, Manning Publications.
5. Internet Engineering Task Force (IETF). (2015). *RFC 7519: JSON Web Token (JWT)*. Available at: https://datatracker.ietf.org/doc/html/rfc7519.
6. Mozilla Developer Network (MDN). (2024). *Express Web Framework (Node.js/JavaScript)*. Available at: https://developer.mozilla.org.
7. Bootstrap Core Team. (2024). *Bootstrap 5 Documentation & Design System*. Available at: https://getbootstrap.com.
8. Chart.js Community. (2024). *Chart.js: Simple yet flexible JavaScript charting for designers & developers*. Available at: https://www.chartjs.org.

APPENDIX

A. User Manual & Local Installation Guide

Step 1: Clone the GitHub Repository

```
git clone https://github.com/gautam7003/spendsmart.git
cd spendsmart
```

Step 2: Configure Environment Variables

Inside `server/.env`, verify your connection string:

```
PORT=5000
MONGODB_URI=mongodb+srv://gautammishra4321_db_user:NjzaeUsISoQAbWxH@cluster0.fhwf4mn.mongodb.net/spendsmart_db?retryWrites=true&w=majority
JWT_SECRET=spendsmart_mca_super_secret_jwt_key_2026
CLIENT_URL=http://localhost:5173
NODE_ENV=development
```

Step 3: Install Dependencies & Run

Open two terminal windows:

Terminal 1 (Backend Server):

```
cd server
npm install
npm run dev
# Server running at http://localhost:5000
```

Terminal 2 (Frontend Client):

```
cd client
npm install
npm run dev
# Vite client running at http://localhost:5173
```

B. Viva Voce Preparation Guide (High-Yield Questions & Answers)

Q1: What is the architectural design pattern used in SpendSmart?

Answer: SpendSmart employs a decoupled **3-Tier Client-Server Architecture** adhering to RESTful principles. The client layer is a single-page application built with React 18 and Bootstrap 5; the application layer is powered by Node.js and Express.js; and the data layer utilizes MongoDB Atlas accessed via Mongoose ODM.

Q2: Why did you use JWT instead of traditional server-side session cookies?

Answer: JSON Web Tokens (JWT) provide a stateless authentication mechanism. The server does not need to store active sessions in memory or query a session store on every incoming request. The client includes the cryptographically signed token in the `Authorization: Bearer <token>` header, making the backend horizontally scalable and decoupled from the client.

Q3: How do you protect against SQL / NoSQL injection attacks?

Answer: By using Mongoose ODM, all queries are parameterized and cast to defined schema types (e.g., String, Number, ObjectId), preventing malicious payload injection. Furthermore, critical routes validate data types before query execution.

Q4: How is password security implemented?

Answer: Passwords are never stored in plaintext. SpendSmart uses `bcryptjs` with a work factor of 10. `bcrypt` automatically generates a random salt for every user, defending against rainbow-table and dictionary attacks. The hash is compared during login using constant-time comparisons via `bcrypt.compare()`.

Q5: How is multi-user data isolation enforced?

Answer: The server never trusts client-supplied user IDs. The `protect` middleware decodes the JWT token, verifies the cryptographic signature, and extracts `req.user._id`. All transaction and budget queries filter strictly by `{ userId: req.user._id }`. Update and delete routes explicitly verify that `record.userId.toString() === req.user._id.toString()`, returning `403 Forbidden` on any unauthorized attempt.

Q6: How does the budget alert mechanism work?

Answer: The budget engine aggregates expense transactions matching the user's category, month, and year using MongoDB aggregation: $$\text{Utilization Percentage} = \left(\frac{\text{Total Category Expenses}}{\text{Monthly Limit}}\right) \times 100$$ If utilization reaches 80%, the system flags a `warning` state with an amber badge. If it reaches or exceeds 100%, it flags an `exceeded` state with a red badge and displays a prominent warning banner across the dashboard.